formalize-paper: an open-source tool for formalizing research papers in Lean on an ordinary computer

Runtian Zhou* Xichen Tang†

Abstract

Lean certifies proofs, but not that a formal statement says what the paper says. We present formalize-paper, an open-source tool that runs in Claude Code or the Codex CLI on an ordinary computer and formalizes a mathematical research paper in Lean 4 with Mathlib. Independent AI reviewers compare every Lean statement with the paper, a sign-off sheet lets a mathematician confirm them without reading Lean, and errors in the paper itself are reported with Lean evidence.

To measure how reliably it reports errors in a paper, we release a benchmark of ten short papers with planted errors and hidden answer keys. On every paper, release 0.3.1 delivered a complete formalization with the goal theorem proved and documented 57 of 61 planted errors; it raised no false alarms, including on ten correct statements written to look wrong. We formalized eight research papers from four fields, one of them on a 2019 MacBook Air with 8 GB of memory, and every project passed an independent kernel check. The tool is at `https://github.com/ZRTMRH/formalize-paper`.

1 Introduction

Formalizing a mathematical research paper means translating its definitions and claims into Lean [dMU21] and proving the resulting statements. Lean checks that these proofs follow from their hypotheses and axioms. It does not establish that the formal statements express the claims printed in the paper. A hypothesis may have been added or altered, a quantifier moved, or incompatible assumptions introduced. An agent may then produce a valid proof of a statement different from the intended one. Paper formalization therefore requires both proof construction and review of the relationship between the formalization and its source.

Our starting point was practical: we wanted to formalize research papers on the computers available to us and obtain results that we could inspect as mathematicians. We therefore built formalize-paper, a tool that runs on an ordinary computer, without needing 32 GB or more of memory, and with a single Claude Code [Ant26] or Codex subscription; the tool itself is free. Users can review the work themselves or delegate specified decisions to the agents, with every decision recorded.

At the scale of a paper, these decisions extend beyond the translation of individual statements. Definitions determine the meaning of subsequent results, intermediate lemmas carry assumptions through an argument, and cited results introduce dependencies on external theory. A paper may also contain an incorrect claim or an incomplete proof. Formalization can expose these problems, but it can also proceed by replacing an argument or proving a modified statement. The resulting project must make clear what was established. A proof of a corrected theorem, a proof conditional on an external result, and a counterexample to the original claim are different mathematical outcomes, each of which can be useful to the reader.

The user-facing materials are part of the workflow. A mathematician should be able to find the formal counterpart of each claim, inspect the reading submitted for approval, and see every decision the agent took on their behalf, with its consequences. They also distinguish a successful proof of a theorem from a verification of the particular argument

*Duke University. Email: `daniel.zhou@duke.edu`
†Duke University. Email: `xichen.tang@duke.edu`

Claim (7) — CHECK: Paper defect (PAPER_DEFECTS row 1): (7) as printed is false for some $0 < x < 2$; approve reading (7) for $x > 0$, its refutation, and the repair $x \geq 2$.

Paper. But it is easy to see that

$$\log\Gamma(x) - 2\log\Gamma\left(\tfrac{1}{2}x + \tfrac{1}{2}\right) \;\leq\; \log[x]! - 2\log[\tfrac{1}{2}x]! \;\leq\; \log\Gamma(x+1) - 2\log\Gamma\left(\tfrac{1}{2}x + \tfrac{1}{2}\right)$$

(no range of x printed; (8) is deduced from it "if $x > 0$").

The Lean, in plain English (blind reading). Two declarations. (a) `display7`: for every real x with $2 \leq x$, both (non-strict) $\log\Gamma(x) - 2\log\Gamma(x/2 + 1/2) \leq \mathrm{logFact}(x) - 2\,\mathrm{logFact}(x/2)$ and $\mathrm{logFact}(x) - 2\,\mathrm{logFact}(x/2) \leq \log\Gamma(x + 1) - 2\log\Gamma(x/2 + 1/2)$. [...] (b) `not_display7_as_printed`: it is NOT the case that for every real x with $0 < x$ both of those same two inequalities hold; i.e. there exists a real $x > 0$ at which at least one of the two inequalities fails. So the two-sided bound is asserted for $x \geq 2$ and refuted as a statement over all $x > 0$.

Differences. Paper defect, PAPER_DEFECTS row 1 (status false, repair substantial). The paper prints (7) with no range and no argument ("it is easy to see") and uses it for every $x > 0$. Read for $x > 0$ it is false: at $x = 1/2$ the middle term is $\log 0! - 2\log 0! = 0$ while the right side $\log\Gamma(3/2) - 2\log\Gamma(3/4) \approx -0.527 < 0$; the left inequality fails at $x = 3/2$. Numerically (7) fails exactly on $0 < x < 1$ and $1 < x < 2$ and holds at $x = 1$ and for $x \geq 2$. The Lean asserts the negation of the printed claim read for $x > 0$ and states the repair for $x \geq 2$ only, which is weaker than the paper on $0 < x < 2$. The range $x \geq 2$ was proposed by the formalizing agent and confirmed by the adjudicator; not yet approved by a human. [...]

Figure 1. One row of the statement sign-off sheet from our run on Ramanujan's proof of Bertrand's postulate [Ram19]: the row for the paper's display (7), marked CHECK. It sets the paper's statement beside a plain-English reading of the Lean, written by a reviewer who did not see the paper, and states the differences the approver must decide. Some technical lines are omitted ([...]). PAPER_DEFECTS is the run's list of errors in the paper; the adjudicator is the agent that rules on the reviewers' findings; the blind reading is written without seeing the paper.

printed in the paper. The agents follow the paper's argument where they can; every place where a Lean proof takes another route is recorded with the reason, and the user can ask for the paper's own argument there or leave the choice to the agent.

Our design builds on Trellis [Peg26a], from which we adapt the one-criterion-per-reviewer design and three review rubrics, and it shares many elements with other recent agent systems for formalizing papers, among them Prove2Me [CML+26], PaperFormalizer [TBS26], LeanAutoformalizationSkills [Arm26], formalize-math-paper [VL26], Theo [SMGS+26] and MerLean [RLQ26] (Section 7). Statement review, staged formalization and separate authoring and reviewing roles are established elements of this line of work; we combine them into one tool for mathematicians and measure it, on a benchmark with hidden answer keys and on research papers. The tool's other features are summarized in the Highlights section of the repository's README [ZT26].

2 Formalization outputs

The central output of a run is a Lean project containing the formal definitions, statements, and completed proofs. The accompanying materials explain how these artifacts relate to the paper. They identify the results covered by the formalization, the assumptions under which they are proved, and any modifications or unresolved claims. This section illustrates how a mathematician can use these materials to review the work.

Reviewing the statements. Before proof construction, the sign-off sheet presents the statements submitted for approval. For each in-scope numbered result, it places the paper's statement beside a natural-language reading of its Lean counterpart and records identified differences. The reading is generated by a reviewer given the Lean but not the paper. Figure 1 shows one row of such a sheet, from our run on Ramanujan's proof of Bertrand's

Using formalize-paper at a glance.

You provide: the paper's LaTeX source, a Lean project with Mathlib, a Claude Code or Codex subscription, and a few answers at the start (which results, which cited theorems to assume, how involved you want to be). You can use it without reading or writing Lean: the sign-off sheet gives a plain-English reading of each statement, with the Lean printed beside it for anyone who wants to check.

You get: Lean in which every result in scope is proved, proved under a stated assumption, corrected, or left unproved with a reason; a plain-language summary and report; a sign-off sheet; a list of the paper's errors, with Lean evidence where possible; and a licence of your choice on the Lean (Apache-2.0 by default), so others can reuse it. Cited results that Mathlib lacks are taken as named hypotheses.

Time and usage: 3–5 h for a two-to-three-page paper; 4–23 h for the research papers run hands-off on our desktop, using 3–15% of a Claude Max 20x weekly limit (Table 1).

Machine: Linux or WSL2, or macOS on a separate branch; 8 GB of memory was enough with one agent at a time (16 GB or more recommended).

To start: clone the repository, install it into a Lean project, try the one-page example first (1–3 h on a desktop), and type `/formalize-paper` (Appendix C).

Confidentiality: the paper's source is sent to the model provider like any other prompt; the project and its outputs stay on your machine, and nothing is published unless you allow it.

postulate [Ram19]: the row for the paper's display (7).

The mathematician can approve the statements or request changes to particular rows. If approval is delegated to an agent, the report records that no human reviewed the statements. A row marked CHECK means that the reading of the Lean differs from the paper's statement in some way: a convention, a deliberate correction of the paper, or an error in the formalization. The approver, the person who signs the sheet, decides which it is, so a CHECK does not by itself mean that anything is wrong. The record of approval names exactly which version of the statements was signed, so any later change to them can be detected.

Inspecting a correction. Figure 2 illustrates how the records present a claim that fails as printed. Lemma 2.3 of benchmark paper bench3 asserts that $x + 1/x \geq 2$ for every real $x \neq 0$. The run proves in Lean that the claim fails at $x = -1$ and proves the corrected statement for $x > 0$.

The subsequent formalization uses the corrected statement, and both the sign-off sheet and the defect report record the change, so a completed proof is never mistaken for a proof of the claim as printed.

Assessing the results. The summary and full report give the status of the in-scope results. They distinguish proofs of the source statements from proofs under additional assumptions or in modified form, and identify claims left unproved. Cited results that remain unproved within the project are represented as explicit named hypotheses, with their use documented. Findings about the paper are accompanied by their supporting evidence; advisory citation notes, which compare each cited result with its openly available source, are presented separately.

The Lean project provides access to the precise statements and proofs behind these descriptions. Its proofs can be checked independently with Lean's `leanchecker`; we did so for every project reported here. The report states which checks were completed and whether any blockers remain, so a successful proof check is not confused with completion of the entire workflow.

Tracing decisions. The decision record identifies who made the recorded choices, which choices were delegated, and the estimated work needed to revisit them. The review records preserve findings and their dispositions, while the run history documents interruptions, recovery actions, and tool patches. Together with the sign-off sheet, these records allow a reader to trace a reported result back to the approved statement, the resulting proof, and any intervening changes.

3 Workflow design

The design organizes paper formalization around three activities: constructing formal statements and proofs, reviewing their relationship to the source, and recording the resulting mathematical outcomes. The authoring agent performs the formalization, separate reviewers examine its

1. Claim as printed.
Bench3, Lemma 2.3 claims that, for every real $x \neq 0$,

$$x + \frac{1}{x} \geq 2.$$

2. Refutation recorded by the run.
The declaration `SumRecip.not_lem_2_3_as_printed` refutes the printed statement. The counterexample is $x = -1$:

$$-1 + \frac{1}{-1} = -2 < 2.$$

The defect record classifies the claim as false as printed.

3. Corrected statement proved by the run.
The declaration `SumRecip.lem_2_3` establishes that, for every real $x > 0$,

$$x + \frac{1}{x} \geq 2.$$

The sign-off sheet identifies the change from $x \neq 0$ to $x > 0$ and asks the reader to approve the repair.

Figure 2. A recorded correction from benchmark paper bench3, Lemma 2.3: the printed claim, its refutation, and the proved repair. Declaration names identify the Lean results; formulas are presented in mathematical notation. The answer key was withheld from the run.

work, and an adjudicator evaluates their findings. Mechanical checks track the project against its approved statements and validation requirements.

This section describes version 0.3.1; the evaluation states the version of each run.

3.1 Task configuration and workflow

A run begins with an agreed scope: the source paper, the in-scope results, the goal results required for the intended outcome, and the permitted use of external mathematical assumptions. The configuration also records the available libraries, review budget, level of human involvement, and authorized external actions. At setup the tool also asks which licence the Lean output should carry (Apache-2.0, the licence of Mathlib, by default) and writes it into the project. Code published without a licence cannot legally be reused by others, and of 312 public Lean formalizations of recent arXiv papers that we checked, 77 (25%) carry none.

The workflow then proceeds through statement translation and approval, proof construction, review and revision, and reporting. A run can wait for its user at three checkpoints: A (confirm the setup), B (sign off the statements) and C (answer open decisions); a hands-off run waits only at A, apart from the stops described in Section 3.6. Before every review round, a mechanical check (Phase 0) builds the project and confirms that the approved statements are unchanged.

The scope distinguishes mathematical goals from supporting work: results outside the agreed scope do not automatically become completion requirements.

3.2 Statement approval and change control

The authoring agent first introduces the in-scope definitions and states the results in Lean, leaving the proofs open. Reviewers examine the hypotheses, conclusions, and definitions that determine the meaning of these statements.

A reviewer who does not see the paper then writes plain-English readings of the Lean, which are presented for approval on the sign-off sheet (Section 2), by the user or by delegation according to the selected involvement level.

The approved statement bundle and the definitions recorded with it are fingerprinted. Subsequent checks detect changes to this recorded material. A planned change begins the reapproval procedure before editing and is submitted for renewed review and approval. An unintended departure from the approved bundle must be restored or otherwise resolved through the prescribed procedure before completion.

This protects what was approved; whether it faithfully expresses the paper is the reviewers' question.

3.3 Proof construction

The authoring agent constructs proofs using Lean, Mathlib [The20] and any other permitted library (by default Tau Ceti [Tau26a]), and the external assumptions permitted by the configuration. It may introduce auxiliary definitions

and lemmas to support the approved targets. This supporting material becomes part of the project and enters the applicable audits.

The proofs are written by the coding agent itself (Claude Code or Codex), following the paper's argument where it can; when a step is hard to formalize, it may prove the same statement another way, and the report says where the route differs. The workflow fixes what must be proved, which assumptions are allowed, when a statement may change and which reviews follow, but not how the agent finds a proof.

A suspected false claim is investigated and, where possible, refuted by a formal counterexample; a proposed correction is recorded separately from the original claim, and any change to an approved statement requires reapproval.

Results outside the goals that remain unproved are listed separately from the proved ones, so the reader can see what was left open.

3.4 Review and adjudication

Reviewers work in separate contexts and examine distinct criteria. These include correspondence between the Lean and its natural-language description, faithfulness to the paper, and possible vacuity. Additional criteria, including substantiveness, depend on the review configuration.

Each finding records the material examined, the concern, and the supporting evidence. The adjudicator evaluates these findings and reruns cited Lean probes where applicable. It assigns a recorded disposition:

- **Adopt**: require a supported change;
- **Reject**: decline the finding with a reason;
- **Defer**: retain the finding without requiring immediate implementation;
- **Question**: identify an unresolved matter requiring clarification.

The authoring agent implements adopted changes, and the affected material is reviewed again. Deferral preserves a finding in the record; it is not a proof that the finding is incorrect. Questions are resolved or escalated according to the workflow's decision rules.

The review budget (light, standard or thorough) sets how many reviewers run and how many rounds may re-check revisions. It never removes the faithfulness reviewers, the review of the statements, the final full review, or Lean's check of the proofs. A smaller budget can only leave more findings unresolved, and these are reported as open rather than counted as passed.

3.5 Validation, reporting, and completion

Agents and code check each other. Twenty-six Python scripts make the mechanical decisions, and they can stop a run: the build and the absence of gaps and extra axioms, the fingerprints of the approved statements, sealed reviewer verdicts that the authoring agent cannot write or alter, and the record of every named hypothesis. They never decide a mathematical question, such as whether a statement is faithful or a step is a defect; there they can only flag, and the reviewers and the adjudicator judge. When one of these scripts misfires, the agent may patch it, but each patch needs a regression test and an independent reviewer's approval and is disclosed in the run record, and the scripts that certify the proofs cannot be patched at all.

Phase 0 (Section 3.1) checks more than the build: no proof may contain a gap (`sorry`), searched for in the compiled proofs as well as in the source text, and every proof may use only Lean's standard axioms. A result the paper cites is stated as an explicit hypothesis, never added as an axiom.

Delivery is refused if an approved statement changed without re-approval, if the Lean changed after the last review round, or if the scripts that check the proofs were altered, even when the mathematics is otherwise done. A late change to the Lean is reviewed again; once two rounds have run after the full review, it is delivered with a disclosure instead.

At the end, the authoring agent writes the summary and the report from the Lean and the run's records. Four fresh reviewers then read them for overstatement, readability, the decisions taken and what the reader is told about trust. Their findings are advice, each answered on record, but every overstatement must be corrected. Whether the run

can be delivered is still decided by the checks above.

3.6 Human involvement and recovery

The configuration selects one of three involvement levels:

- **Involved**: the user confirms the setup (A), signs off the statements (B) and answers the open decisions (C);
- **Statements-only**: the user confirms the setup and signs off the statements; later decisions are delegated, but any change to an approved statement comes back to the user;
- **Hands-off**: the user confirms the setup; everything after is delegated to the agents.

At every level, the run stops for the user only when a goal result looks false or unprovable (for a reason other than missing library theory), when it would take an outward action not allowed at setup, when something only the user can fix arises (a login, a usage limit, permissions), or when the paper's meaning is genuinely ambiguous on a point that matters. Every decision taken for the user is recorded, with what it would cost to revisit.

Interruptions, recoveries and every tool patch are recorded in the run history.

Two assumptions remain. Whether a statement is faithful to the paper is a judgement, made by reviewers or by people. And the consistency checks catch an agent that skips or misreports a step, but they are not designed against deliberate forgery of the project's history, which we have not observed in any run. Both are separate from the validity of the proofs, which Lean's kernel checks independently.

4 Evaluation setup

We evaluate formalize-paper on the questions a mathematician would ask before relying on it. Does it catch errors in a paper? We measure this on a benchmark of ten short papers with planted errors and hidden answer keys (Section 5.1). Does it find errors in real research papers (Section 5.2)? Does a run finish without help, and when does it need the user (Section 5.3)? What does a complete formalization of a research paper look like, and how can it be checked (Section 5.4)? And how long does a run take, and on what computer (Section 5.5)?

4.1 Papers and runs

We report two groups of runs. **Benchmark:** the ten benchmark papers, run hands-off on release 0.3.1, bench1–5 on a desktop computer (Linux under WSL2, with 26–28 GB available to Linux) and bench6–10 on a laptop reporting 47 GB to Linux. **Research papers:** eight papers, by Le, Shi and Wei [LSW26], Hefty, Horn, King and Pfender [HHKP25], Bradač [Bra26], De Medts [DM26], Ramanujan [Ram19], Pisani [Pis26], Zweiman [Zwe26] and Matijasevick [Mat26]. We chose the papers from different fields: the research papers come from combinatorics (Le–Shi–Wei, Hefty–Horn–King–Pfender, Bradač), number theory (Ramanujan, Pisani, Matijasevick), algebra (De Medts) and analysis (Zweiman), and the benchmark papers cover number theory, graph theory, inequalities, combinatorics, linear algebra, analysis and group theory. Table 1 lists every run with its version, agent, machine and time; among them Bradač's paper was run statements-only on a virtual machine with 17 GB, and Ramanujan's on a 2019 MacBook Air with 8 GB. All runs used medium effort: Claude Code with Claude Opus 5.5, and Codex with GPT-6.1-Sol.

4.2 The benchmark

The benchmark consists of ten short papers written by Claude agents, bench1–10, each approximately two to three pages long. The papers contain planted issues involving false statements, boundary cases, vacuous assumptions, proof errors, citation discrepancies, conflicting definitions, and redundant hypotheses. Each also contains a decoy: a correct statement intended to appear suspicious.

The paper-generation agents were not given the tool or its protocol. Each paper was accompanied by an answer key describing the planted items, their witnesses or supporting arguments, and their intended corrections. A separate agent checked the keys through mathematical reasoning and computation.

Table 1. Runs reported in this paper. All were hands-off except Bradač (statements-only) and L∞ (a version before 0.2.0, with the approver answering decisions). Time is wall-clock from Checkpoint A to delivery, waits included. Linux is the desktop under WSL2; Mac is the 2019 MacBook Air. Memory is what Linux (or macOS) had available. 20x: estimated share of a Max 20x weekly limit (Section 4.6); –, not estimated. Example note: the one-page example shipped with the tool.

Run	Version	Agent, machine, memory	Time	20x
bench1	0.3.1	Claude, Linux, 28 GB	4 h 33 min	–
bench2	0.3.1	Claude, Linux, 26–28 GB	11 h 13 min[a]	–
bench3	0.3.1	Claude, Linux, 26–28 GB	3 h 57 min	–
bench4	0.3.1	Claude, Linux, 26 GB	4 h 58 min[b]	–
bench5	0.3.1	Claude, Linux, 26 GB	4 h 17 min[b]	–
bench6–10	0.3.1	Claude, laptop, 47 GB	2 h 43 min–3 h 33 min	–
Le–Shi–Wei (L∞)	pre-0.2.0	Claude, Linux, 24 GB	48 h 31 min	22%
Hefty–Horn–King–Pfender	0.3.0	Claude, Linux, 28 GB	22 h 32 min	15%
Bradač	0.3.0	Claude, VM, 17 GB	29 h 26 min	–
De Medts	0.3.1	Claude, Linux, 28 GB	3 h 51 min	3.5%
De Medts	0.3.1	Codex, Linux, 28 GB	4 h 00 min	–
Ramanujan	0.3.1	Claude, Linux, 28 GB	4 h 39 min	3%
Ramanujan	0.3.1	Claude, Mac, 8 GB	25 h 06 min	5%
De Medts	0.3.1	Codex, Mac, 8 GB	23 h 26 min[c]	–
Pisani	0.3.1	Claude, Linux, 28 GB	12 h 01 min	5%
Zweiman	0.3.1	Claude, Linux, 28 GB	9 h 25 min	8%
Matijasevick	0.3.1	Claude, Linux, 28 GB	6 h 52 min	9%
Example note	0.3.1	Claude, Linux, 28 GB	1 h 19 min	–
Example note	0.3.1	Codex, Linux, 28 GB	2 h 31 min	–
Example note	0.3.1	Claude, Mac, 8 GB	7 h 08 min	–
Example note	0.3.1	Codex, Mac, 8 GB	3 h 46 min	–

[a] Includes about 6 h of mutation tests run inside a tool patch. [b] Includes an outage of about 1 h 15 min when the disk filled. [c] About 8 h of work; the rest was an avoidable stall in a tool patch, a power-off and network interruptions.

One author additionally performed a manual spot-check of 12 key entries: one planted item from each of the ten benchmark papers and two decoys. Each selected entry was checked against the corresponding passage at the cited location in the paper's source. All 12 entries were confirmed. The papers, answer keys and scoring records are in the folder `benchmark/` of the repository [ZT26], at commit `85c0a06`; the runs scored in Section 5.1 used the tool at tag `v0.3.1`.

4.3 How findings are scored

The unit of scoring is a planted key item, not a theorem or a Lean declaration. Each item receives one label from what the reader of the run's deliverables is told:

- **Found**: the delivered summary or report presents it as a defect.
- **Noted**: the summary or report states the fact but does not call it a defect, for example as a reading choice or an unneeded hypothesis.
- **Missed**: it appears nowhere in what the reader receives.

A decoy reported as a defect counts as a false alarm, and findings outside the key are checked against the paper; none of them was a false alarm. Section 5.1 (Table 2) gives the Found rate and the documented rate (Found and Noted together), over all items and without the items that are only unneeded hypotheses, since a statement with an unneeded hypothesis is true, just weaker than it could be. The authors

assigned the final labels; Appendix B describes the papers, and Section 5.1 the independent re-scorings.

4.4 Human input

A run's involvement level says who is asked to approve and decide; it does not mean that a hands-off run received no human input. We therefore record each kind of human input separately: statement approvals, answers to stops, operating instructions and mathematical hints. bench6–10 and Bradač's run received, after Checkpoint A, a short note on how the tooling behaves; the runs record that no decision changed because of it. The human approval of statements, in Bradač's run, was made by one of the authors with an AI-written reading aid.

For the research papers, we separate formal evidence, a Lean proof, from informal checks: reading the source, computing examples, and comparing with a cited work. Every research-paper finding reported here was checked against the paper and the Lean by a separate agent, and by hand by one of the authors.

Training-data contamination. A paper may be in the model's training data. We therefore count a finding as evidence of detection only for papers that appeared after the model's training cutoff (June 2026): the papers of Le–Shi–Wei, De Medts, Zweiman, Pisani and Matijasevick, all from October 2026. The papers of Hefty–Horn–King–Pfender (October 2025), Bradač (May 2026) and Ramanujan (1919) appeared before the cutoff, so they are reported as case studies.

4.5 How results are checked

For each result in scope, the report says whether it was proved as stated, proved under an extra assumption, proved in a corrected form, or left unproved; whether the run delivered is recorded separately. Three checks support these statements:

- **Kernel replay.** Outside the run, the project is rebuilt and every result is re-checked by Lean's own kernel (`leanchecker`). This confirms the proofs. A result deliberately left unproved, with a `sorry`, also passes the replay, so such results are always listed separately; a result may be left unproved only by a recorded decision, and a goal result never.
- **Axiom check.** Phase 0 confirms that every proof uses only Lean's standard axioms.
- **Statement review.** The reviewers and the sign-off sheet check that each Lean statement says what the paper says.

The benchmark measures whether the planted errors are reported, not whether every statement is translated correctly.

4.6 Measuring time and memory

Time is measured from Checkpoint A to delivery; waits for a person, usage-limit pauses and outages are reported separately. Each memory figure says what it measures: the whole machine, one process, or the kernel replay. Token counts are measured from the session transcripts and priced at the Claude Opus 5.5 API list prices, as an API-equivalent measure. To express usage as a share of a subscription, we use one direct measurement: the run on Matijasevick's paper, the only user of a Claude Team premium seat at the time, used 18% of that seat's weekly limit. Taking a premium seat's limit as about half of a Max 20x plan's, that is about 9% of a 20x week; for the other runs we estimate the share in proportion to their API-equivalent cost. Our runs differ in paper, version, review budget, machine and human involvement, and the errors planted in one paper are not independent of each other. We therefore report what we observed, not general error rates or controlled comparisons (Section 8).

5 Results

5.1 Catching errors in a paper

On release 0.3.1, across all ten benchmark papers, the runs recorded 46 of the 61 planted items as Found, documented 11 others as Noted, and missed four. Table 2 gives the results by paper. Documented recognition was therefore $57/61$ (93.4%), while the Found rate was $46/61$ (75.4%).

Table 2. Planted errors by paper, all runs on release 0.3.1. Each paper also contains one correct statement written to look wrong (a decoy); none was reported as a defect.

Paper	Planted	Found	Noted	Missed
bench1	6	5	0	1
bench2	6	5	0	1
bench3	7	4	3	0
bench4	6	5	1	0
bench5	6	4	2	0
bench6	6	5	1	0
bench7	6	4	1	1
bench8	6	4	1	1
bench9	6	5	1	0
bench10	6	5	1	0
All ten	61	46	11	4

Leaving out the nine items that are only unneeded hypotheses, which the tool by design reports without calling them defects, it documented 49 of the remaining 52 items (94.2%) and found 46 of them (88.5%) as defects. bench1–5 were scored by a fresh independent agent per run, and bench6–10 by one of the authors, under the same rules.

Recorded observations. Of the eleven Noted items, eight are unneeded hypotheses, which the tool by design reports without calling them defects. The other three were reported as reading choices (two) and as a misquotation outside the agreed scope (one). Independent re-scorings agreed item by item: an agent given only the keys and the delivered records re-scored bench6–10, and the other author re-scored bench6 and bench7.

Evidence for the findings. Of the 46 errors found, 35 come with Lean evidence produced by the run: 29 printed statements refuted in Lean, and 6 shown in Lean to be vacuous. Of the other 11, seven are errors in the paper's proofs described in the report, three are misquoted cited results, each compared with its source by the citation check, and one is a definition whose wording and formula disagree.

The four misses. Three are errors inside proofs of true statements (in bench2, bench7 and bench8). In each case the run proved the statement by its own route and reported it as proved as printed, so the faulty step in the paper's proof went unnoticed; the seven other planted errors in proofs were reported. Finding another proof does not check the paper's own argument (Section 6). The fourth miss is an unneeded hypothesis in bench1 ($p \geq 5$ where every odd prime would do): the reviewers noticed it, but it did not reach the delivered report. Every false, vacuous or misquoted statement in the ten papers was reported.

False alarms. None of the ten decoys was reported as a defect, and no reported defect turned out to be a false alarm.

5.2 Verification of research papers

Table 3 summarizes the research-paper cases. As in Section 4.5, we distinguish results proved as stated from results proved under an extra assumption, proved in a corrected form, or left unproved, and describe each finding by its effect on the paper's results. A cited result taken as given is stated in Lean as an explicit hypothesis. Bracketed keys such as [ASW] are the papers' own reference labels. We first discuss the five papers that appeared after the model's training cutoff (Le–Shi–Wei, De Medts, Zweiman, Pisani, Matijasevick), and then the three earlier papers (Bradač, Hefty–Horn–King–Pfender, Ramanujan). Following Section 4.4, the earlier papers are case studies: their findings are reported, not counted as evidence of detection.

Le–Shi–Wei (L∞). The paper of Le, Shi and Wei [LSW26] is 35 pages long and has 67 numbered items: theorems, lemmas, claims, facts, corollaries, definitions and constructions. Every one of them has a Lean counterpart. Each theorem, lemma, claim, fact and corollary, including the main Theorems 1.1, 1.2 and 4.1, is stated and proved, and each definition and construction is defined, with no unfinished proof (`sorry`) and nothing assumed beyond Lean's standard axioms. The formalization is public (version 1.0, `https://github.com/ZRTMRH/le-shi-wei-linf-lean`); its file `CERTIFICATE.md` sets each main theorem's Lean statement beside the paper's, so a reader can check them without reading the

Table 3. Research-paper outcomes. Modified statements, external assumptions, and unfinished claims are distinguished from proofs of the printed statements.

Paper	Formalization outcome	Findings recorded
Le–Shi–Wei [LSW26]	All numbered results stated and proved; no outside assumption.	No incorrect statement or proof step found.
De Medts [DM26]	All numbered results stated and proved; no outside assumption.	No incorrect statement or proof step found; one citation remark limited to the version of Deré's article that was accessible.
Zweiman [Zwe26]	All numbered results stated; all proved except Lemma 2.4 (Garloff–Wagner, taken as given), and Conjecture 1.2, stated only; the cited PF characterization assumed where used.	No incorrect statement or proof step found; two citation remarks on attribution and theorem numbering.
Pisani [Pis26]	All numbered results stated; all proved except Theorems 3 and 4, quoted results the formal proofs do not need; no outside assumption.	No incorrect statement or proof step found; one citation remark: the general construction goes beyond the official solutions.
Matijasevick [Mat26]	All numbered results stated; all proved, Proposition 7.2 in corrected form, under stated readings, except that Remark 6.8 assumes Gauss's three-triangular-numbers theorem.	The empty-set exception in Proposition 7.2 is corrected by requiring nonemptiness in the affected clause; the author's formalization makes the same restriction.
Bradač [Bra26]	All numbered results stated; all proved except Theorem 1.2 and Lemma A.1, quoted results that the paper's proofs do not use. The cited Lemmas 2.5 and 2.6 are also proved.	Three small proof gaps and one typo; no incorrect statement found.
Hefty–Horn–King–Pfender [HHKP25]	The main Theorems 1.2 and 1.3 proved, assuming cited tail bounds; Theorem 5.1 stated, its upper bound proved assuming Shearer's bound.	An invalid middle inequality in Lemma 4.1 when its exponent is negative, and five typos.
Ramanujan [Ram19]	Bertrand's postulate and all numbered displays proved, (7) in corrected form and (18) in a stated reading; no outside assumption.	Display (7) false as printed for some $0 < x < 2$, proved for $x \geq 2$; an error in deducing (8) from (7).

Lean project. No incorrect statement or proof step was found. Some intermediate claims are, by design, steps inside proofs by contradiction (in the proofs of Lemmas 4.5, 4.10, 4.13, 4.20 and 4.40): their hypotheses describe the situation being ruled out, so they can never hold together, exactly as the argument intends. The formalization records this; it is not an error in the paper. Where a hypothesis is not needed by the Lean proof, it was kept so that the statement matches the paper, and where a Lean proof takes another route than the paper's, the route is recorded and in most cases proves the same step at the same strength. An independent audit after the run judged the formalization valid (Section 5.4): main statements faithful, hypotheses satisfiable, and every difference from the paper a disclosed convention that changes no meaning.

De Medts. Every numbered result was proved without outside assumptions, including Theorem 5.1 and both directions of Lemma 2.3. No incorrect statement or proof step was found. One small citation remark: page 1 says Deré mentions that the converse also holds for nilpotent complex Lie algebras up to dimension 7, while the version of Deré's article that the run could access says his meth-

ods "can be applied" there; the journal version may read differently. A second run, using Codex instead of Claude Code, reached the same outcome (Section 5.5).

Zweiman. Every numbered result was formalized. All were proved, including Theorem 1.3(a)–(c), except two: Lemma 2.4, the Garloff–Wagner lemma that the paper cites, which was taken as given, and Karp's Conjecture 1.2, which was formalized as a statement only. The conjecture's case of positive Pólya-frequency (PF) sequences is proved as the last part of Theorem 1.3. The cited PF characterization (Definition 1.1, [ASW]) was also taken as given where the paper uses it: for that last part of Theorem 1.3, for Corollary 1.5, and for the PF claim in Example 4.1. No incorrect statement or proof step was found. Two small citation remarks: the paper cites [ASW, Ed] for the direction "only negative roots implies PF", but Edrei's paper attributes this direction to Schoenberg (1948) and proves the converse; and the result cited as [Ka12, Theorem 1] is Theorem 1 in the POMI version but Theorem 2 in arXiv:1203.1482v1, so it may help to say which version the numbering refers to.

Pisani. The main result, Theorem 2, was proved exactly as stated without outside assumptions. All other results were proved except the two quoted theorems of Ballot–Williams, Theorems 3 and 4. The steps of Corollary 5 and Lemma 12 that use them in the paper were proved directly, so the formal proofs do not depend on those quoted results. No incorrect statement or proof step was found. One small citation remark (in the paper's notation): the construction for general odd g ("in accordance with [sols]") goes beyond the official solutions, which give only the case $\{a, b\} = \{1, 2\}$ with $r = 1/2 - \sqrt{n}$, so the paper may wish to say that the general version is its own.

Matijasevick. All 34 numbered results were formalized, and 33, including Theorems A–D, were proved, a few under stated readings, that is, recorded interpretations of the paper's wording; for instance, Theorem D's uniqueness is read up to the order of the factors, as in Theorem 7.8(a). The 34th, Remark 6.8, was proved with Gauss's three-triangular-numbers theorem taken as given. Theorem 6.1 and Remark 6.2 were also proved without the cited result on generalized Pell equations. One small remark: Proposition 7.2 does not hold as printed when $S = \varnothing$: $\mathrm{End}(\varnothing)$ contains the zero map even though $0 \notin \varnothing$, so its parenthetical exclusion of zero fails in this case. Adding nonemptiness to the affected clause gives the proved correction. The author's own Lean formalization makes the same restriction. The main results are unaffected.

Bradač. Every numbered result was stated in Lean and proved, including the main Theorem 1.1, except the two results quoted from the literature that the paper's proofs do not use, Theorem 1.2 of Ma, Shen and Xie and the Lovász local lemma, Lemma A.1, which were stated and taken as given. The cited expander-mixing and Alon–Rödl lemmas, Lemmas 2.5 and 2.6, were also proved, so Theorems 1.1, 1.3 and 2.10 are proved without them. These unconditional versions of Theorems 1.1 and 1.3 were added after the human approval of the statements (Section 3.2). No statement was found to be incorrect. Three small gaps in proofs were recorded, each easily filled and none affecting a statement. In Lemma 3.1, the count of vertices of $D(s-2, 2)$ appears to count ordered edges rather than non-edges; the lemma follows by keeping N of its vertices. The proof of Lemma 2.11 omits the concluding step: x_1 is orthogonal to $y_1, \ldots, y_{t+1}$ and hence zero. The proof of Lemma 2.7 uses $\Delta \geq h$, which is not assumed but may be after replacing h by $\min(h, \Delta)$. That proof also has a small typo: z_0 where z_i is intended. The main results are unaffected.

Trellis formalized the same paper independently [Peg26a] (from the paper's first arXiv version, and with a different model). The five theorems Trellis treats as main results all have counterparts in our run, each proved and, by our reading of the two Lean statements, equivalent to those of Trellis; for Theorem 1.5 the two read "$a = o(s)$" differently, and the equivalence needs a diagonal argument that neither side proves in Lean. Trellis reports no errors in the paper, but its Lean repairs all three gaps: Lemma 3.1 in the same way as ours, Lemma 2.7 by adding $h \leq \Delta$ as a hypothesis that it proves where the lemma is used, and Lemma 2.11 by supplying the missing step. This independently confirms all three.

Hefty–Horn–King–Pfender. Theorems 1.2 and 1.3 were proved, taking as given the standard tail bounds the paper uses: the Chernoff–Hoeffding inequality (Theorem A.2) and its hypergeometric version (in the proof of Lemma 3.1). McDiarmid's inequality (Theorem A.4) is also taken as given, though no proof uses it. Theorem 5.1, whose proof the paper sketches, was stated but not proved; its upper bound was proved assuming Shearer's bound. The paper was also one of the test cases of Trellis [Peg26a], whose formalization [Peg26c], finished with an operator-directed repair, proves such cited inequalities rather than assuming them.

In Lemma 4.1, the middle inequality in the displayed chain does not hold when its exponent is negative. In that range both outer bounds exceed 1, and the lemma is only used where the exponent is positive; stating the two bounds separately resolves the problem. Two small typos are in the proof of Lemma 4.1, $E(G_R)$ for $E(G_B)$ at line 635 of the arXiv LaTeX source and $10\varepsilon_1$ for $15\varepsilon_1$ at line 649; three further minor typos are $\pi_R(I)$ for $\pi_B(I)$ at line 417, the undefined $N_3(v_1)$ for N_{v_1} at line 553, and ℓ_r for ℓ_R at lines 664–665. The main results are unaffected.

Ramanujan. Bertrand's postulate was proved along Ramanujan's displays (8)–(15) for $x \geq 162$, with a finite check below 162; Stirling's formula was replaced by other facts about Γ, and the change is disclosed. Display (18) was proved under a stated reading. Display (7) is false as printed for some $0 < x < 2$: it fails at $x = \frac{1}{2}$ (the failure is proved in Lean), and it was proved for $x \geq 2$, which is all that (8) and (9) need (Figure 1 shows its row on the sign-off sheet). Deducing (8) from (7) "if $x > 0$" is therefore an error; nothing else is affected. As a classic paper may be in the model's training data, these findings are a case study, not evidence of detection.

Citation check. Each run also checks, as advisory notes, how the paper uses its citations. When a cited source is openly available, for example on arXiv, it checks that the source states each cited result in the form used, at the cited place, and that the bibliography entries are correct and up to date. On the seven research papers that have citation notes (all except Le–Shi–Wei), it reached 123 of the 166 places where a paper invokes a cited result (74%) and 59 of the 82 cited works (72%), including 35 of the 36 on arXiv; the works it could not reach were mostly books and articles behind paywalls. The check does not yet run under Codex; running it there is planned.

Table 4. Benchmark runs on 0.3.1. *Stops* are recorded halts of the workflow after Checkpoint A; the three here were bugs in the run's own checking scripts, each repaired by the run with an independently reviewed patch, so no human answer was needed. Time is from Checkpoint A to delivery, waits included ([†]from the setup commit).

Run	Stops	Human answers	Time
bench1	1	0	4 h 33 min
bench2	2	0	11 h 13 min
bench3	0	0	3 h 57 min
bench4	0	0	4 h 58 min
bench5	0	0	4 h 17 min
bench6	0	0	2 h 43 min
bench7	0	0	3 h 18 min
bench8	0	0	3 h 09 min[†]
bench9	0	0	2 h 52 min[†]
bench10	0	0	3 h 33 min[†]

5.3 Finishing without help

Every run in Table 1 delivered: the ten benchmark papers on 0.3.1 and every research-paper run. Delivery means that the run completed; it is not a claim that every statement in the paper was proved unchanged.

Stops after setup. A stop is a recorded halt of the workflow: the run either resolves it itself, for example by repairing one of its own checking scripts, or waits for a person. All stops were resolved by the run itself except two (Table 4 for the benchmark): the Pisani run needed one human ruling on a tool fault, and the Mac run on Ramanujan's paper one answer to a file-permission prompt. None of the stops concerned the mathematics.

Other human input was the statement approval in Bradač's run and the approver's decisions in the L∞ run, on a version before 0.2.0.

5.4 Complete formalizations of research papers

In all eight papers, every assumption is explicit. For Le–Shi–Wei, De Medts and Ramanujan, every numbered result was proved (Ramanujan's display (7) in corrected form) with nothing assumed beyond Lean's standard axioms; for Bradač and Pisani, the only results not proved are quoted results that the proofs do not use. In the other three, the gaps are results the papers cite from the literature, which by default enter as named hypotheses rather than being re-proved: the Garloff–Wagner lemma and a cited characterization (Zweiman), Gauss's three-triangular-numbers theorem (Matijasevick), and tail bounds and Shearer's bound (Hefty–Horn–King–Pfender, whose Theorem 5.1 was stated but only its upper bound proved). The following checks bear on how far these formalizations can be trusted.

Independent formalizations of the same paper. For Bradač, Trellis's independent formalization [Peg26b] agrees with ours on the five main theorems, and its Lean repairs all three gaps we report, although neither its records nor the Trellis paper flag them as gaps (Section 5.2). Matijasevick's paper comes with the author's own Lean formalization. After our run, an AI agent compared the two by reading their statements and definitions. It judged the main Theorems A–D equivalent and found two differences that bear on faithfulness. In Theorem 6.3(b), the phrase "such a g" admits two readings, both true: our run took the weaker one and disclosed it, while the author's Lean states the stronger one, which the paper's proof establishes and which is therefore most likely the intended meaning. The stronger reading follows from the weaker in two lines, as a reviewer argued on paper, but this step is not in our Lean. Choices like this are where the user's involvement matters: at the involved level the run puts such a reading to the user (Checkpoint C). In a hands-off run the adjudicator decides it, and the decision is recorded in the run's decisions document, with what it would cost to revisit, so the user can reverse it afterwards. In Remark 6.8, the author's Lean leaves out the clause that relies on Gauss's three-triangular-numbers theorem, while ours states it with that theorem taken as given. Neither formalization contradicts the paper. The comparison was made by reading and is not proved in Lean.

Independent audits. After delivery, a separate agent re-read five of the research-paper runs against the paper and the Lean, without changing anything. It judged the Le–Shi–Wei and Pisani formalizations valid, and those of Hefty–Horn–King–Pfender, Zweiman and Matijasevick valid with caveats: for Hefty–Horn–King–Pfender, that the main theorems rest on cited results taken as given and that Theorem 5.1 is only partly proved, both disclosed in its report; for Zweiman and Matijasevick, the wording of the delivered documents, not the mathematics.

Kernel replay. The Lean of every benchmark run and of every research-paper project passed a kernel replay, checked module by module; on the research-paper projects a module needed 2.1–7.8 GB.

A 100-page draft. We also ran the tool, hands-off, on a draft in progress of about 100 pages by colleagues, not yet public. It formalized all 76 numbered items and proved every theorem, lemma, proposition, corollary and claim among them, with only Lean's standard axioms; the proofs were complete about 6 h after setup. When the authors revised the draft, the new and changed results, among them two new lemmas, were added to the same formalization and proved as well.

5.5 Time, memory and cost

Time. Table 1 gives each run's time from Checkpoint A to delivery, waits included. The benchmark runs took 2 h 43 min to 4 h 58 min each, except bench2 (11 h, about 6 h of it tests run inside a tool patch). The research-paper runs on the desktop took 4–12 h (Hefty–Horn–King–Pfender: 22.5 h). Bradač's 29 h included about 16 h waiting for the statement approval. The L∞ run's 48.5 h, on a version before 0.2.0, included about 16 h of model activity; the rest was waiting for the approver's 13 decisions and two outages. As a share of a Max 20x week (Section 4.6), the research-paper runs used from about 3% (Ramanujan) to 22% (L∞; Table 1).

Memory. A single run peaked at 8.7–9.7 GB on the desktop (Matijasevick 8.7 GB, the Codex run on De Medts

9.7 GB). Three runs at once on the 28 GB desktop crashed it.

An 8 GB laptop. A 2019 MacBook Air (Intel Core i5-8210Y, two cores, 8 GB) delivered Ramanujan's paper with one agent at a time, in about 25 h of work against 4 h 39 min on the desktop, and its Lean passed kernel replay. It was slower mainly because 8 GB means one agent at a time, and partly because of its two-core 7 W processor; at API list prices the run cost about $115 against $75 on the desktop, or about 5% of a Max 20x week against 3%. This is one run, but it suggests that an ordinary computer suffices.

Agents and platforms. Under Codex (GPT-6.1-Sol), De Medts delivered in 4 h with the same outcome as under Claude Code. On macOS (the `macos` branch), the Mac also delivered the example note under both Claude Code and Codex, and, under Codex, De Medts's paper with the same outcome, after about 8 h of work.

Cost. We also ran Trellis ourselves on L∞, on the same desktop with the same model (Claude Opus 5.5, medium effort); both runs produced a complete formalization using only the standard axioms. Counting every model call in the transcripts, and correcting Trellis's own cost ledger, which counts each call about 1.8 times on average, our run (on a version before 0.2.0) cost about $500 at API list prices and Trellis's about $18,000–28,000, roughly 36–56 times more.

6 Failure modes that shaped the design

Rigid checks stall finished work. Hard-coded checks on the delivered documents sometimes refused to deliver runs whose mathematics was already complete, over a formatting mismatch or an unforeseen case. Now only the checks on the formalization itself can block delivery (that the proofs build with no gaps or extra axioms, that the approved statements are unchanged, and that the final Lean was reviewed or, after the review limit, disclosed); a problem in the documents becomes a note for the agent to resolve.

Code that guards agents needs guarding too. Python checks are reliable on the cases they were written for, but on an unforeseen case they can stall a run or raise a false alarm: 10 of the 19 stops after setup in the benchmark and research-paper runs were bugs in the checks themselves. So code and agents guard each other (Section 3.5): the agents may repair a check, except those that certify the proofs, which the patch tool refuses to modify, and each repair needs a regression test and a fresh reviewer's approval. The agents proposed 12 patches in five runs; reviewers rejected 15 of the 24 versions they examined, 9 patches were applied (six after at least one rejection) and 3 were withdrawn. None touched the proof checks. In bench1, for example, the sign-off check did not recognise entries headed "`## Theorem 1.1`", the form the tool's own reading prompt asks for. A reviewer rejected the first fix because it also made keys that differ only in spacing match; a second reviewer approved the revised fix after reproducing the bug and comparing old and new code on many inputs, and the stop was closed 46 minutes after it opened. Only two stops needed a person, and one of them shows the same lesson one level up: the patch tool's own guard mistook the sign-off script for a signed record and refused to patch it (since fixed).

A different proof can hide an error in the paper's proof. The run proves each statement its own way, so a faulty step in the printed proof can go unnoticed; this caused three of the four benchmark misses. Reading the paper's proofs step by step is left for future work.

7 Related work

Paper-level systems. Trellis [Peg26a] organizes a formalization around a dependency graph, with separate roles for proving, verifying and adjudicating, and freezes the formal statements once a human approves them; our one-criterion reviewers and three rubrics are adapted from it. SHEAF [Fre26, DGG$^+$26] likewise reviews and locks target statements before proving. Other re-

cent agent systems for papers include LeanAutoformalizationSkills [Arm26], which freezes statements once the author approves them; PaperFormalizer [TBS26], whose reviewer agent checks every statement and its proof in natural language; and formalize-math-paper [VL26], an agent skill with a name close to ours. Prove2Me [CML+26], Theo [SMGS+26] and MerLean [RLQ26] check statements by translating the Lean back into natural language, as our sign-off reading does; Prove2Me, like us, hides the source from the translator, and freezes statements once a human approves them. LeanFlow [MGNK26] checks statements before proving and reports token use per paper. Our paper-faithfulness checklist also adapts LeanMarathon's cheap-falsification checks [ZSS+26], LeanFlow's encoding-bridge criterion and TauCetiReview's correctness rubric [Tau26b]. M2F [WMM+26] formalizes mathematical literature at scale with statements fixed before proofs, and Numina Fuse [ACQ+26] supports interactive, human-supervised formalization of papers.

Faithfulness and proving. A Lean proof that compiles does not show that its statement is the intended one; "Beyond Compilation" [ZGCM+26] measures this gap, and Kontorovich [Kon25] proposes that mathematicians take responsibility for checking formal statements. Our sign-off sheet lets them do so without reading Lean. Learned provers such as Seed-Prover [CGH+25], Aristotle [ABB+25], Goedel-Prover [LTL+25] and DeepSeek-Prover-V2 [RSS+25] construct proofs of formal statements. Our proofs are written by general coding agents (Claude Code or Codex), and our contribution is the workflow around the proving: which statements to prove, what was assumed or changed, and what the finished project establishes.

8 Conclusion and future work

formalize-paper turns a mathematics paper into a Lean formalization with a record of how each result relates to the paper, and reports where the paper itself is wrong. On a benchmark of ten papers with hidden answer keys it documented 57 of 61 planted errors (46 as defects) with no false alarms, and it formalized eight research papers from four fields, one of them on an 8 GB laptop; every project passed an independent kernel replay.

These results suggest several next steps. The benchmark papers are short and written by agents; larger benchmarks, built from research papers and from planted changes to formal statements, would measure detection more broadly, and new papers remain the best test, since older ones may be in a model's training data. A study with mathematicians outside this project would measure how well the sign-off sheet lets them check statements without reading Lean. A reviewer that checks each step of the paper's own proofs would address the misses described in Section 6. Results that rest on cited theorems kept as named hypotheses could be made unconditional by formalizing those theorems. Finally, controlled runs of the same paper across versions, models and machines would turn our time and cost figures into a comparison.

Acknowledgements

We thank Domagoj Bradač, Mateo Matijasevick and Anthony Pisani, who checked our descriptions of their papers, and the other authors of the papers studied here for their kind replies. We thank Viet Hung Le, Ruilin Shi and Fan Wei for the collaboration on their paper that became our main case study, and Wesley Pegden, whose Trellis design and review rubrics we adapted. We thank Fan Wei and Colleen Robles for their advice.

Use of AI

Formalize-paper is itself an AI system, and AI was used throughout this work. The tool's code, protocol and tests were written largely by Claude (Anthropic, through Claude Code with Claude Opus models) under the authors' direction and review. The formalizations reported here were produced by the tool, that is, by Claude agents (and, in the Codex runs, by OpenAI models). Their independent audits, the benchmark papers and answer keys, and the scoring of bench1–5 and the re-scoring of bench6–10 were also carried out by separate AI agents; one of the authors scored bench6–10, checked a sample of the keys by hand,

and checked by hand every reported finding in a named paper. The authors chose the design, the papers and the experiments, and checked the reported findings. Claude also helped refine this report: it improved the exposition, checked figures against the run records, and assisted with LaTeX typesetting. The authors reviewed and verified all revisions and take full responsibility for the content.

References

[ABB+25] Tudor Achim, Alex Best, Alberto Bietti, et al. Aristotle: IMO-level automated theorem proving, 2025. arXiv:2510.01346.

[ACQ+26] Justin Asher, Yichuan Cao, Ruichen Qiu, Pietro Pellegrino, and Jia Li. Numina Fuse: Interactive autoformalization in Lean 4, 2026. arXiv:2610.33860.

[Ant26] Anthropic. Claude Code. `https://claude.com/claude-code`, 2026.

[Arm26] Scott Armstrong. LeanAutoformalizationSkills. `https://github.com/scottnarmstrong/LeanAutoformalizationSkills`, 2026.

[Bra26] Domagoj Bradač. Off-diagonal Ramsey numbers, 2026. arXiv:2605.28793.

[CGH+25] Luoxin Chen, Jinming Gu, Liankai Huang, et al. Seed-Prover: Deep and broad reasoning for automated theorem proving, 2025. arXiv:2507.23726.

[CML+26] Shuze Chen, Kunal Marwaha, Xiaoyang Lu, Henry Yuen, and Tianyi Peng. Prove2Me: An open collaborative platform for scaling math formalization, 2026. arXiv:2608.28433.

[DGG+26] Bin Dong, Guoxiong Gao, Bin Guo, Zeming Sun, Bin Wu, and Song-Yan Xie. Constructing rational curves via jets on projective varieties with non-nef canonical bundle, 2026. arXiv:2609.28465. Appendix B, by B. Dong, G. Gao, Z. Sun and B. Wu, reports the Pharos system and the SHEAF formalization.

[DM26] Tom De Medts. A 9-dimensional nilpotent Lie algebra isomorphic to its complex conjugate admitting no real form. arXiv:2610.13707, 2026.

[dMU21] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[Fre26] FrenzyMath. SHEAF: Scalable hierarchical engine for autonomous formalization. `https://github.com/frenzymath/SHEAF`, 2026. Accessed 2026-10-30.

[HHKP25] Zion Hefty, Paul Horn, Dylan King, and Florian Pfender. Improving $R(3, k)$ in just two bites. arXiv:2510.19718v3, 2025.

[Kon25] Alex Kontorovich. The shape of math to come, 2025. arXiv:2510.15924.

[LSW26] Viet Hung Le, Ruilin Shi, and Fan Wei. A complete dichotomy for L_∞ permutation removal. arXiv:2610.28271, 2026.

[LTL+25] Yong Lin, Shange Tang, Bohan Lyu, Jiayun Wu, Hongzhou Lin, Kaiyu Yang, Jia Li, Mengzhou Xia, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-Prover: A frontier model for open-source automated theorem proving, 2025. arXiv:2502.07640.

[Mat26] Mateo Matijasevick. Retriangular numbers and affine maps between polygonal numbers. arXiv:2610.36284, 2026.

[MGNK26] Lazar Milikic, Simon Guilloud, Khanh Nguyen, and Viktor Kuncak. LeanFlow: A case study in workflow-driven Lean autoformalization, 2026. arXiv:2607.20503.

[Peg26a] Wesley Pegden. (Auto)formalization is supposed to be easy: Trellis process semantics for spelling out rigorous proofs, 2026.

arXiv:2606.09674. Code: https://github.com/wpegden/trellis.

[Peg26b] Wesley Pegden. offdiagonal: a Trellis formalization of Bradač's off-diagonal Ramsey paper. https://github.com/wpegden/offdiagonal, 2026. Commit e845f47.

[Peg26c] Wesley Pegden. twobites: a Trellis formalization of Hefty, Horn, King and Pfender's paper. https://github.com/wpegden/twobites, 2026.

[Pis26] Anthony Pisani. On the open problem in the 2023 Simon Marais Mathematics Competition. arXiv:2610.30658v2, 2026.

[Ram19] S. Ramanujan. A proof of Bertrand's postulate. *Journal of the Indian Mathematical Society*, 11:181–182, 1919.

[RLQ26] Yuanjie Ren, Jinzheng Li, and Yidi Qi. MerLean: An agentic framework for autoformalization in quantum computation, 2026. arXiv:2602.16554.

[RSS+25] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. DeepSeek-Prover-V2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025. arXiv:2504.21801.

[SMGS+26] Arshia Soltani Moakhar, Iman Gholami, Max Springer, Mahdi JafariRaviz, and MohammadTaghi Hajiaghayi. Beyond the library: An agentic framework for autoformalizing research mathematics, 2026. arXiv:2606.31134.

[Tau26a] Tau Ceti Project. Tau Ceti: an AIs-welcome Lean library downstream of Mathlib. https://github.com/TauCetiProject/TauCeti, 2026.

[Tau26b] Tau Ceti Project. TauCetiReview: review rubrics and machinery for Tau Ceti. https://github.com/TauCetiProject/TauCetiReview, 2026.

[TBS26] Jiyuan Tan, Jose Blanchet, and Vasilis Syrgkanis. PaperFormalizer: Author-in-the-loop autoformalization of research papers in Lean, 2026. arXiv:2610.31810.

[The20] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381, 2020.

[VL26] The-Anh Vu-Le. formalize-math-paper: an agent skill for formalizing a mathematics research paper in Lean 4 with Mathlib. https://github.com/vltanh/formalize-math-paper, 2026.

[WMM+26] Zichen Wang, Wanli Ma, Zhenyu Ming, Gong Zhang, Kun Yuan, and Zaiwen Wen. M2F: Automated formalization of mathematical literature at scale, 2026. arXiv:2602.17016.

[ZGCM+26] Ke Zhang, Patricio Gallardo Candela, Sudhir Murthy, Yi Xie, Zhi Wang, and Maziar Raissi. Beyond compilation: Evaluating faithful natural-language-to-Lean statement formalization, 2026. arXiv:2606.31002.

[ZSS+26] Yuanhe Zhang, Yuekai Sun, Taiji Suzuki, Jason D. Lee, and Fanghui Liu. LeanMarathon: Toward reliable AI co-mathematicians through long-horizon Lean autoformalization, 2026. arXiv:2606.05400.

[ZT26] Runtian Zhou and Xichen Tang. formalize-paper. https://github.com/ZRTMRH/formalize-paper, version 0.3.1, 2026.

[Zwe26] Henry Zweiman. A proof of Karp's conjecture on the zeros of Toeplitz determinants of rising factorial series. arXiv:2610.18895, 2026.

A Design details

What each reviewer checks. Each reviewer is a fresh agent that may not edit the project and checks one criterion. *Correspondence*: the Lean statement means what its English description says, once every definition it uses is unfolded. *Paper-faithfulness*: for each numbered result, the Lean is at least as strong as the paper's statement and no stronger than the paper justifies. Definitions are compared word by word with the paper, and where the statements differ the reviewer tries to build a separating example. *Non-vacuity*: each definition, set of hypotheses and main theorem has a satisfying example; a suspected contradiction must be proved in Lean. *Substantiveness*: each statement has the right strength; a hypothesis is called unused only after the reviewer deletes it and recompiles. *Prior art*, only for a library contribution: whether Mathlib already has the result. Statement rounds use the first three; the final full review uses the first four.

How a reviewer reports. Every item in scope gets a verdict: Pass, Fail, Ambiguous (two readings of the paper) or not examined. An item left out counts as not examined, never as passed, and a Pass must say what was compared or run. A Fail says whether the Lean or the paper is at fault; a fault in the paper goes to the defect record.

How the adjudicator decides. A fresh adjudicator, never the agent that wrote the Lean, reproduces every Fail itself, reading the paper rather than the Lean's comments. It cannot alone reject a Fail on correspondence, faithfulness, non-vacuity or statement strength: a second fresh reviewer re-examines the item, and if it also fails, the finding cannot be rejected. An Ambiguous verdict becomes a question.

The freeze and the proof check. The fingerprint of an approved statement covers every definition it depends on, so a changed definition is caught even when the statement's text is unchanged. The proof check walks every declaration in the project and allows only Lean's three standard axioms. A result may be left unproved only by a recorded decision, never a goal result, and any proof that uses it is itself reported as unproved.

How delegated decisions are taken. The agent follows the adjudicator's written recommendation. Without one it takes the conservative option: keep the paper's statements and hypotheses, even unneeded ones; never generalize or strengthen a statement; take a missing result as a named hypothesis stating exactly what the paper invokes, unless only definitions and easy lemmas are missing; and on a reading choice take the weaker reading. Each delegated decision is final and recorded with the cost of reversing it.

B The benchmark

Papers. bench1 polite numbers; bench2 repeated degrees in graphs; bench3 a sum–reciprocal inequality; bench4 gcd of the numbers $2^n - 1$; bench5 divisibility and coprimality in subsets of $\{1, \ldots, 2n\}$; bench6 quadratic residues modulo an odd prime; bench7 nilpotent and idempotent real matrices; bench8 the iteration $x_{n+1} = \sqrt{2 + x_n}$; bench9 involutions in groups of even order; bench10 rational roots of integer polynomials. Each is two to three pages of LaTeX by a fictional author, with a definition, five to ten numbered results, a short bibliography of real sources, and a true goal theorem.

The writing of the papers and keys, the independent key checks, the hand check and the scoring rules are described in Section 4 and, in full, in `benchmark/README.md` of the repository [ZT26], which also holds the papers, keys and score files.

Planted errors by type.

Type	bench1–5	bench6–10
Numbered result false as printed	3	2
Missing hypothesis at an edge case	3	3
Vacuous statement	3	3
Unneeded hypothesis	4	5
Cited result stated more strongly than its source	5	5
Typo in a statement	4	3
Error inside a proof only	5	5
Definition clashing with its later use	2	0
Off-by-one in a range	2	4
Total	31	30

Blinding. The keys were kept on a machine that ran no

formalization and were opened only after each run had delivered. The runs saw only the paper's LaTeX source and PDF.

C Kernel replay and reproduction

After delivery, each project was re-checked outside the run with Lean's external type checker, `leanchecker`, which re-verifies the compiled declarations rather than trusting the elaborator that produced them; it uses no model tokens. We ran it one module at a time, in a fresh clone of each project's branch. Every project reported in this paper passed; the largest, Le–Shi–Wei's, has 13 modules, each of which needed about 6.3 GB.

Installation, a first run, the independent check of a run's proofs, and the hardware and account requirements are described in `README.md` and `INSTALL.md` of the repository [ZT26] (release 0.3.1; macOS on the `macos` branch). The benchmark is in `benchmark/` at commit `85c0a06`.